

Linked List Questions - Complete Study Guide

Question 1: Memory Allocation Type

Question: Linked list is considered as an example of which type of memory allocation?

Options:

- A) Dynamic
- B) Static
- C) Compile time
- D) Heap

Answer: A) Dynamic

Explanation: Linked lists use dynamic memory allocation because nodes are created at runtime using operators like `new` or `malloc()`. Memory is allocated as needed during program execution, not at compile time. While the memory technically comes from the heap, "Dynamic" is the most accurate characterization of the allocation type.

Question 2: Space Complexity for Deleting a Linked List

Question: What is the space complexity for deleting a linked list?

Options:

- A) $O(1)$
- B) $O(n)$
- C) $O(n^2)$
- D) None

Answer: A) $O(1)$

Explanation: Deleting a linked list requires constant extra space regardless of the list size. You traverse the list once, deleting each node, but you only need a temporary pointer variable. No additional data structures proportional to n are needed.

Question 3: Time Complexity of Rotating Linked List

Question: Rotating a linked list by some places clockwise will take a time complexity of?

Options:

- A) $O(1)$
- B) $O(n)$
- C) $O(n^2)$
- D) None

Answer: B) $O(n)$

Explanation: To rotate a linked list, you need to:

1. Traverse to find the new tail (rotation point)
2. Adjust pointers to make it circular temporarily
3. Break the link at the appropriate position

This requires traversing the list, which takes $O(n)$ time in the worst case.

Question 4: Space Complexity of Rotating Linked List

Question: Rotating a linked list by some places clockwise will take a space complexity of?

Options:

- A) $O(1)$
- B) $O(n)$
- C) $O(n^2)$
- D) None

Answer: A) $O(1)$

Explanation: Rotation only requires a few temporary pointer variables to rearrange the links. No additional space proportional to the list size is needed.

Question 5: Function Functionality Analysis

Question: What is the functionality of the following piece of code?

```
cpp
```

```
int function() {  
    Node* temp = tail->prev;  
    tail->prev = temp->prev;  
    temp->prev->next = tail;  
    size--;  
    return temp->data;  
}
```

Options:

- A) Return the element at the tail of the list but do not remove it
- B) Return the element at the tail of the list and remove it from the list
- C) Return the last but one element from the list but do not remove it
- D) Return the last but one element at the tail of the list and remove it from the list

Answer: D) Return the last but one element at the tail of the list and remove it from the list

Explanation:

- `temp = tail->prev` gets the second-to-last node
 - The function then re-links pointers to remove `temp` from the list
 - `size--` decrements the count
 - Returns `temp->data` (the second-to-last element)
 - This effectively removes the second-to-last node
-

Question 6: Doubly Linked List Properties

Question: Which of the following is false about a doubly linked list?

Options:

- A) We can navigate in both the directions
- B) It requires more space than a singly linked list
- C) Both
- D) None

Answer: D) None

Explanation: Both statements A and B are TRUE:

- A doubly linked list allows bidirectional traversal (true)
 - It requires more space due to an extra pointer per node (true)
 - Since both are true, neither is false, so the answer is "None"
-

Question 7: Circular Linked List Search Function

Question: What is the functionality of the following piece of code?

```
cpp

void function(Node* head, int data) {
    int flag = 0;
    if(head != nullptr) {
        Node* temp = head->next;
        while((temp != head) && !(temp->data == data)) {
            temp = temp->next;
            flag = 1;
            break;
        }
    }
    if(flag)
        cout << "success" << endl;
    else
        cout << "fail" << endl;
}
```

Options:

- A) Print success if a particular element is not found
- B) Print fail if a particular element is not found
- C) Print success if a particular element is equal to 1
- D) None

Answer: D) None

Explanation: This code has a bug. The `flag = 1; break;` inside the while loop means flag is set to 1 and the loop breaks immediately after the first iteration, regardless of whether the element is found. The logic is flawed and doesn't correctly implement search functionality.

Question 8: Counting Elements in Circular Linked List

Question: How do you count the number of elements in the circular linked list?

Options:

- A) Starting length at 1, traverse until temp != head
- B) Starting length at 0, traverse until temp != null
- C) Starting length at 0, traverse until temp != head && temp != nullptr
- D) None

Answer: A) Starting length at 1, traverse until temp != head

Explanation: For a circular linked list:

- Start with `length = 1` (counting the head node)
- Begin from `head->next`
- Loop while `temp != head` (circular condition)
- This correctly counts all nodes in a circular list

Option B is wrong because in a circular list, `next` never points to `null`.

Question 9: Circular vs Normal Linked List

Question: What differentiates a circular linked list from a normal linked list?

Options:

- A) You cannot have the 'next' pointer point to null in a circular linked list
- B) It is faster to traverse the circular linked list
- C) You may or may not have the 'next' pointer point to null in a circular linked list
- D) Head node is known in circular linked list

Answer: A) You cannot have the 'next' pointer point to null in a circular linked list

Explanation: The defining characteristic of a circular linked list is that the last node's `next` pointer points back to the head instead of being null. This creates a circular structure. Options B and D are not defining differences.

Question 10: Inserting n Elements in Sorted Order

Question: What is the worst case time complexity of inserting n elements into an empty linked list, if the linked list needs to be maintained in sorted order?

Options:

- A) $\Theta(n)$
- B) $O(n \log n)$
- C) $O(n^2)$
- D) None

Answer: C) $O(n^2)$

Explanation:

- Inserting the 1st element: $O(1)$
- Inserting the 2nd element: $O(2)$ worst case
- Inserting the nth element: $O(n)$ worst case
- Total: $1 + 2 + 3 + \dots + n = n(n+1)/2 = O(n^2)$

Each insertion requires traversing the list to find the correct position.

Question 11: Searching in Singly Linked List

Question: In the worst case, the number of comparisons needed to search a singly linked list of length n for a given element is?

Options:

- A) $\log(2*n)$
- B) $(n/2)$
- C) (n)
- D) $(\log(2*n)-1)$

Answer: C) (n)

Explanation: In the worst case, the element is at the end or not present, requiring traversal of all n nodes. Linear search through a linked list has $O(n)$ comparisons in the worst case.

Question 12: Rearrange Function Output

Question: The following function rearranges elements. Given input 1,2,3,4,5,6,7, what will be the output?

```
cpp
```

```

void rearrange(Node* list) {
    Node* p, q;
    int temp;
    if (list == NULL || list->next == NULL) {
        return;
    }
    p = list;
    q = list->next;
    while (q != null) {
        temp = p->value;
        p->value = q->value;
        q->value = temp;
        p = q->next;
        q = p != NULL ? p->next: NULL;
    }
}

```

Options:

- A) 1,2,3,4,5,6,7
- B) 2,1,4,3,6,5,7
- C) 1,3,2,5,4,7,6
- D) 2,3,4,5,6,7,1

Answer: B) 2,1,4,3,6,5,7

Explanation: The function swaps adjacent pairs:

- Swap positions 0 and 1: 2,1
- Swap positions 2 and 3: 4,3
- Swap positions 4 and 5: 6,5
- Position 6 (7) remains unchanged (no pair)

Question 13: Sorted List Verification Function

Question: What does function f return 1 for?

cpp

```
int f(item *p) {  
    return (  
        (p == NULL) ||  
        (p->next == NULL) ||  
        ((p->data <= p->next->data) && f(p->next))  
    );  
}
```

Options:

- A) Not all elements in the list have the same data value
- B) The elements in the list are sorted in non-decreasing order of data value
- C) The elements in the list are sorted in non-increasing order of data value
- D) None of them

Answer: B) The elements in the list are sorted in non-decreasing order of data value

Explanation: This recursive function returns 1 (true) if:

- List is empty (base case)
- Single element (base case)
- Current element $\leq$ next element AND the rest of the list is also sorted

This checks for non-decreasing order (sorted ascending, allowing duplicates).

Question 14: Finding 8th Element Complexity

Question: What are the time complexities of finding 8th element from beginning and 8th element from end in a singly linked list? ($n > 8$)

Options:

- A) $O(1)$ and $O(n)$
- B) $O(1)$ and $O(1)$
- C) $O(n)$ and $O(1)$
- D) $O(n)$ and $O(n)$

Answer: A) $O(1)$ and $O(n)$

Explanation:

- **8th from beginning:** Traverse 8 nodes = $O(1)$ constant time (fixed small number)

- **8th from end:** Need to reach position $(n-7)$, requires traversing most/all of the list = $O(n)$

Alternatively, you'd need to traverse the entire list to find the length first, then traverse again.

Question 15: Finding Intersecting Node

Question: What is the worst case time complexity of finding the intersecting node from two intersecting linked lists?

Options:

- A) $\Theta(n*m)$, where m, n are lengths of given lists
- B) $\Theta(n^2)$, where $m > n$
- C) $\Theta(m+n)$, where m, n are lengths of given lists
- D) $\Theta(\min(n, m))$

Answer: C) $\Theta(m+n)$

Explanation: The optimal algorithm:

1. Traverse both lists to find their lengths ($m + n$ operations)
2. Advance the pointer in the longer list by $|m-n|$ positions
3. Traverse both lists together until they meet

Total: $O(m) + O(n) + O(\min(m,n)) = O(m+n)$

Question 16: Pointers Affected in Doubly Linked List Insertion

Question: In a doubly linked list, the number of pointers affected for an insertion operation will be?

Options:

- A) 5
- B) 0
- C) 1
- D) None of these

Answer: D) None of these

Explanation: For inserting a node in a doubly linked list:

- New node's next pointer (1)

- New node's `prev` pointer (1)
- Previous node's `next` pointer (1)
- Next node's `prev` pointer (1)

Total: **4 pointers** are modified, which is not listed in options A, B, or C.

Question 17: O(1) Operations with Head and Tail Pointers

Question: Which operation CANNOT be implemented in O(1) time with head and tail pointers?

Options:

- A) Insertion at the front of the linked list
- B) Insertion at the end of the linked list
- C) Deletion of the front node of the linked list
- D) Deletion of the last node of the linked list

Answer: D) Deletion of the last node of the linked list

Explanation:

- **A, B, C:** All O(1) with head/tail pointers
 - **D:** In a singly linked list, to delete the last node, you need to update the second-to-last node's `next` pointer. Without a `prev` pointer, you must traverse from head to find it = O(n)
-

Question 18: Length-Dependent Operation

Question: For list F->1->2->3->L, which operation depends on the length of the linked list?

Options:

- A) Delete the first element of the list
- B) Interchange the first two elements of the list
- C) Delete the last element of the list
- D) Add an element at the end of the list

Answer: C) Delete the last element of the list

Explanation: With pointers F (first) and L (last):

- **A:** O(1) - direct access via F

- **B:** $O(1)$ - access via F and $F \rightarrow \text{next}$
 - **C:** $O(n)$ - must traverse to find second-to-last node
 - **D:** $O(1)$ - direct access via L
-

Question 19: Doubly vs Singly Linked List Efficiency

Question: Which operation is performed more efficiently by doubly linked list than by linear linked list?

Options:

- A) Deleting a node whose location is given
- B) Searching an unsorted list for a given item
- C) Inserting a node after the node with a given location
- D) Traversing the list to process each node

Answer: A) Deleting a node whose location is given

Explanation:

- **Doubly linked:** $O(1)$ - can directly access prev and next nodes
- **Singly linked:** $O(n)$ - must traverse from head to find previous node

Options B, C, and D have similar complexity in both list types.

Question 20: Deleting a Node in Doubly Linked List

Question: Which code correctly deletes node X from a doubly linked list?

```
cpp
class Node {
    int Value;
    struct Node *Fwd;
    struct Node *Bwd;
};
```

Options:

- A) `X->Bwd->Fwd = X->Fwd; X->Fwd->Bwd = X->Bwd;`
- B) `X->Bwd.Fwd = X->Fwd ; X.Fwd->Bwd = X->Bwd;`
- C) `X.Bwd->Fwd = X.Bwd; X->Fwd.Bwd = X.Bwd;`

- D) $X \rightarrow Bwd \rightarrow Fwd = X \rightarrow Bwd; X \rightarrow Fwd \rightarrow Bwd = X \rightarrow Fwd;$

Answer: A) $X \rightarrow Bwd \rightarrow Fwd = X \rightarrow Fwd; X \rightarrow Fwd \rightarrow Bwd = X \rightarrow Bwd;$

Explanation: To delete node X:

1. Make X's previous node point forward to X's next node
2. Make X's next node point backward to X's previous node

Option A correctly uses $(\rightarrow)$ for pointer access and properly bypasses node X.

Question 21: Information in Doubly Linked List Nodes

Question: Which information is stored in a doubly-linked list's nodes?

Options:

- A) Value of node
- B) Address of next node
- C) Address of the previous node
- D) All of the above

Answer: D) All of the above

Explanation: A doubly linked list node contains:

- Data/value field
 - Pointer to the next node ((next))
 - Pointer to the previous node ((prev))
-

Question 22: Recursive Print Output

Question: What will be the output for list 1->2->3->4->5->6?

cpp

```

void solve(node* start) {
    if(start == NULL)
        return;
    cout << start->data << endl;
    if(start->next != NULL)
        solve(start->next->next);
    cout << start->data << " ";
}

```

Options:

- A) 123456
- B) 135531
- C) 135135
- D) 246135

Answer: B) 135531

Explanation:

- First prints: 1, 3, 5 (every other node going forward)
- Recursion unwinds, prints again: 5, 3, 1
- Output: 1 3 5 5 3 1

Question 23: Pointers Modified in Middle Insertion

Question: Insertion of an element at the middle of a linked list requires the modification of how many pointers?

Options:

- A) 2
- B) 1
- C) 3
- D) 4

Answer: A) 2

Explanation: To insert a new node between two existing nodes:

1. Set new node's next to point to the next node (1 pointer)
2. Set previous node's next to point to the new node (1 pointer)

Total: 2 pointer modifications

Question 24: Infeasible Algorithm for Linked List

Question: Which algorithm is NOT feasible to implement in a linked list?

Options:

- A) Linear Search
- B) Merge Sort
- C) Insertion Sort
- D) Binary Search

Answer: D) Binary Search

Explanation: Binary search requires $O(1)$ random access to middle elements, which linked lists don't provide (they have $O(n)$ access). Linear search, merge sort, and insertion sort can all work with sequential access.

Question 25: Sum Calculation Output

Question: What will be the value of "sum" for input 1->2->3->4->5?

```
cpp
void solve(ListNode* root) {
    int sum = 0;
    while (root != NULL) {
        sum += root->val;
        root = root->next;
    }
    cout << sum << endl;
}
```

Options:

- A) 15
- B) 20
- C) 5
- D) 1

Answer: A) 15

Explanation: The function sums all values: $1 + 2 + 3 + 4 + 5 = 15$

Question 26: Resolve Function Purpose

Question: What does the resolve function do?

```
cpp

ListNode* resolve (ListNode* head) {
    ListNode* prev = NULL;
    if(head == NULL) return head;
    if(head->next == NULL) return head;

    ListNode* curr = head->next;
    while(head != NULL) {
        head->next = prev;
        prev = head;
        head = curr;
        if(curr != NULL) {
            curr = curr->next;
        }
    }
    return prev;
}
```

Options:

- A) Returns the original linked list
- B) Returns the linked list after reversing it
- C) Returns a linked list containing elements at odd indices only
- D) None

Answer: B) Returns the linked list after reversing it

Explanation: This is the standard iterative linked list reversal algorithm. It reverses the direction of all next pointers and returns the new head (which was the original tail).

Question 27: Recursive Print Function

Question: What does fun1 do for a given Linked List?

```
cpp
```

```
void fun1(node* head) {  
    if(head == NULL)  
        return;  
    fun1(head->next);  
    cout << head->data << endl;  
}
```

Options:

- A) Prints all nodes of linked lists
- B) Prints all nodes of linked list in reverse order
- C) Prints alternate nodes of linked list
- D) Prints alternate nodes in reverse order

Answer: B) Prints all nodes of linked list in reverse order

Explanation: The function recurses to the end of the list first, then prints while unwinding the recursion stack. This results in reverse order printing.

Question 28: Linked List vs Array

Question: Which points are true about Linked List when compared with array?

Options:

- A) Arrays have better cache locality that can make them better in terms of performance
- B) It is easy to insert and delete elements in Linked List
- C) Random access is not allowed in a typical implementation of Linked Lists
- D) All of the above

Answer: D) All of the above

Explanation: All statements are true:

- Arrays have contiguous memory (better cache locality)
 - Linked lists allow $O(1)$ insertion/deletion when position is known
 - Linked lists require $O(n)$ traversal for random access (no indexing)
-

Question 29: Doubly Linked List Reversal

Question: What should be the modified linked list after calling fun on 1<-->2<-->3<-->4<-->5<-->6?

cpp

```
void fun(node **head_ref) {
    node *temp = NULL;
    node *current = *head_ref;
    while (current != NULL) {
        temp = current->prev;
        current->prev = current->next;
        current->next = temp;
        current = current->prev;
    }
    if(temp != NULL)
        *head_ref = temp->prev;
}
```

Options:

- A) 2<-->1<-->4<-->3<-->6<-->5
- B) 5<-->4<-->3<-->2<-->1<-->6
- C) 6<-->5<-->4<-->3<-->2<-->1
- D) 6<-->5<-->4<-->3<-->1<-->2

Answer: C) 6<-->5<-->4<-->3<-->2<-->1

Explanation: The function swaps `prev` and `next` pointers for each node, effectively reversing the doubly linked list. The head becomes the tail and vice versa.

Question 30: Reverse Function Missing Statement

Question: What statement is missing at the end of the reverse function?

cpp

```
void reverse(node** head_ref) {
    node* prev = NULL;
    node* current = *head_ref;
    node* next;
    while (current != NULL) {
        next = current->next;
        current->next = prev;
        prev = current;
        current = next;
    }
    /*ADD A STATEMENT HERE*/
}
```

Options:

- A) `*head_ref = prev;`
- B) `*head_ref = current;`
- C) `*head_ref = next;`
- D) No missing line

Answer: A) `*head_ref = prev;`

Explanation: After the loop completes:

- `current` is NULL (end of original list)
- `prev` points to the last node of the original list (new head)
- We need to update the head reference to point to `prev`

Therefore, `*head_ref = prev;` is the correct statement.

Summary of Key Concepts

Time Complexities

- **Access nth element:** $O(n)$
- **Search:** $O(n)$
- **Insert at known position:** $O(1)$
- **Delete at known position:** $O(1)$ for doubly, $O(n)$ for singly
- **Insert n elements in sorted order:** $O(n^2)$

Space Complexities

- **Most operations:** $O(1)$ auxiliary space
- **Recursive operations:** $O(n)$ stack space

Key Differences

- **Singly vs Doubly:** Doubly can traverse backward, uses more memory
- **Circular vs Normal:** Circular has no NULL pointers, last node points to head
- **Linked List vs Array:** Arrays have better cache locality and $O(1)$ random access; linked lists have $O(1)$ insertion/deletion

Important Algorithms

- **Reversal:** Use three pointers (prev, current, next)
- **Cycle Detection:** Use Floyd's cycle detection (fast/slow pointers)
- **Finding Middle:** Use two-pointer technique
- **Merge Two Sorted Lists:** Compare and link nodes